

# Adding Items to an Azure Cosmos DB Container

## Summary:-

In this lab, you will create an Azure Cosmos DB database and container, add JSON documents, and query the stored data using SQL-like queries. You will configure a partition key, insert multiple documents, execute queries to retrieve and filter data, and understand how Azure Cosmos DB organizes and searches NoSQL documents.

## Let's Start:-

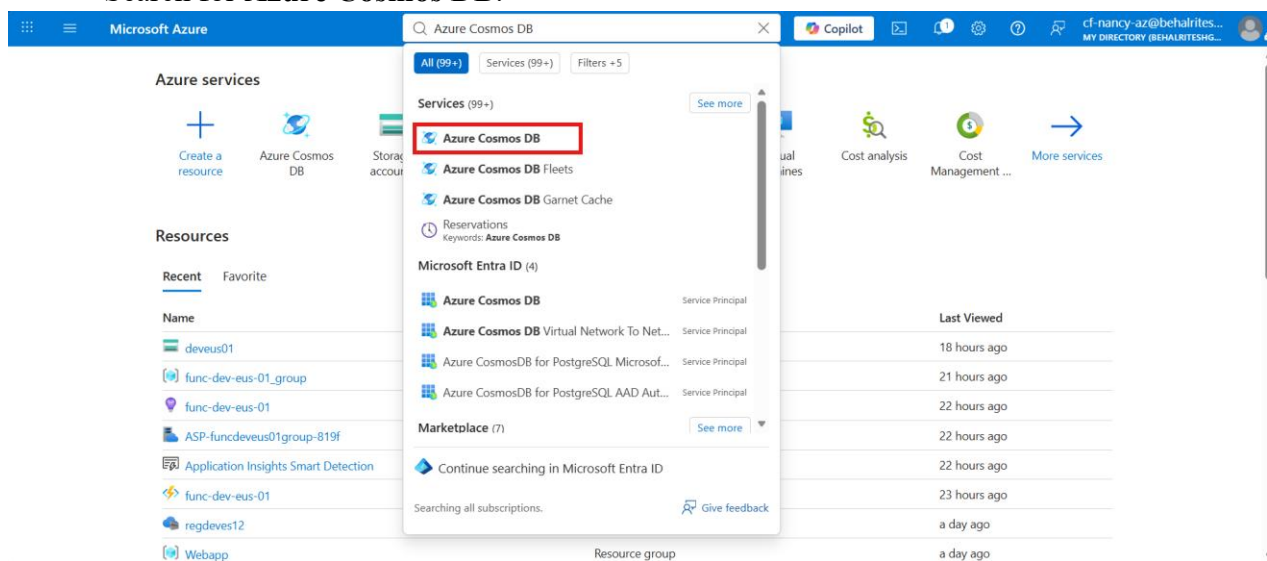
After completing this lab, you will be able to:

- Create an Azure Cosmos DB account and database.
- Create a container with a partition key.
- Add JSON documents to a container.
- Execute SQL queries to retrieve and filter documents.
- Search documents using SQL operators in Azure Cosmos DB.

## Lab Steps:-

### Step 1 – Create an Azure Cosmos DB account.

- Sign in to the Azure portal.
- Search for **Azure Cosmos DB**.



- Select **Azure Cosmos DB**. Click Create.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home

### Azure Cosmos DB

My Directory (behalritesg@mail.onmicrosoft.com)

How many Azure Cosmos DB accounts do I have? Identify non-compliant Azure Cosmos DB accounts using ARG. Filter Azure Cosmos DB accounts by resource group.

Create Restore Manage view Refresh Export to CSV Open query Assign tags Add to service group Group by none

Filter for any field... Subscription equals all Type equals all Resource Group equals all Location equals all Add filter

No Azure Cosmos DB accounts to display

Create a globally distributed, multi-model, fully managed database using API of your choice. Or try it for free, up to 20k RU/s, for 30 days with unlimited renewal.

Create

Try now

Showing 1 - 0 of 0. Display count: auto

Give feedback

- Under **Recommended**, select **Azure Cosmos DB for NoSQL**.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > Azure Cosmos DB

### Create an Azure Cosmos DB account

#### Which API best suits your workload?

Azure Cosmos DB is a fully managed NoSQL and relational database service for building scalable, high performance applications. [Learn more](#)

To start, select an API to create a new account. The API selection cannot be changed after account creation. To use multiple APIs, you can create separate accounts with different APIs as needed.

Recommended APIs Other APIs

#### Azure Cosmos DB for NoSQL

Azure Cosmos DB's core, or native API for working with documents. Supports fast, flexible development with familiar SQL query language and client libraries for .NET, JavaScript, Python, and Java.

Create Learn more

#### Azure DocumentDB (with MongoDB compatibility)

Agent-ready, open-source, cloud agnostic database, fully managed on Azure. MongoDB drivers, tools, and app code work without rewrites. Migrate your existing databases to Azure DocumentDB and save 40%+.

Create Learn more

Give Feedback

Help improve this page

- Click **Create**.
- Configure the following settings:
  - **Workload Type:** Learning
  - **Subscription:** Select your Azure subscription.
  - **Resource Group:** Select your existing resource group or create a new.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > Azure Cosmos DB > Create an Azure Cosmos DB account

## Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

**Basics** Global distribution Networking Backup Policy Security Tags Review + create

Azure Cosmos DB is a fully managed NoSQL and relational database service for building scalable, high performance applications. Go to production starting at \$24/month per database, multiple containers included. [Learn more](#)

**Project Details**

Choose a workload type that best aligns with your goals. This helps us provide an optimized starting point for your Azure Cosmos DB account setup. Don't worry—no matter which workload type you choose, you can customize each setting to fit your needs or stick to the defaults provided.

Workload Type \*

Best for beginners. Low cost, easy setup, and quick exploration to learn Azure Cosmos DB.

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription \*

Resource Group \*

[Create new](#)

**Instance Details**

[Review + create](#) [Previous](#) [Next: Global distribution](#) [Feedback](#)

- **Account Name:** Enter a unique name.
- **Region:** Select East US/ East US 2.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > Create an Azure Cosmos DB account

## Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

**Instance Details**

Account Name \*

Configure availability zone settings for your account. You cannot change these settings once the account is created.

Availability Zones ☐ Enable ☒ **Disable**

Location \*

Available locations are determined by your subscription's access and availability zone support (if that is enabled). If you don't see or cannot select your desired location, please open a support request for region access.  
[Click here for more details on how to create a region access request](#)

**Capacity mode** ☒ **Provisioned throughput** ☐ Serverless

Request units per second (RU/s) are pre-configured and billed hourly, offering guaranteed throughput with two modes—**Autoscale**, which dynamically adjusts within a 1-10x range for varying workloads, and **Manual**, with fixed RU/s for steady, predictable traffic.

[Review + create](#) [Previous](#) [Next: Global distribution](#) [Feedback](#)

- Under **Capacity Mode**, select **Provisioned Throughput**.
- Enable **Apply Free Tier Discount**.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > Azure Cosmos DB > Create an Azure Cosmos DB account

## Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

[Click here for more details on how to create a region access request](#)

**Info** Unsure of your traffic patterns? Explore Azure Cosmos DB's serverless offering if you're just starting out and skip capacity planning by only paying for the request units (RUs) you consume. [Learn more](#)

Capacity mode

☒ **Provisioned throughput**  
Request units per second (RU/s) are pre-configured and billed hourly, offering guaranteed throughput with two modes— **Autoscale**, which dynamically adjusts within a 1-10x range for varying workloads, and **Manual**, with fixed RU/s for steady, predictable traffic.

☐ **Serverless**  
Ideal for intermittent or unpredictable traffic. Billed only for consumed RU/s, scaling on-demand without capacity planning or minimum charges.  
[Learn more about capacity mode](#)

With Azure Cosmos DB free tier, you will get the first 1000 RU/s and 25 GB of storage for free in an account. You can enable free tier on up to one account per subscription. Estimated \$64/month discount per account.

Apply Free Tier Discount ☒ **Apply** ☐ Do Not Apply

Limit total account throughput ☒ Limit the total amount of throughput that can be provisioned on this account  
**Info** This limit will prevent unexpected charges related to provisioned throughput. You can update or remove this limit after your account is created.

[Review + create](#) [Previous](#) [Next: Global distribution](#) [Feedback](#)

- Click **Next**.
- On the **Global Distribution** tab, leave all settings at their default values.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > Azure Cosmos DB > Create an Azure Cosmos DB account

## Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Basics Global distribution Networking Backup Policy Security Tags Review + create

Configure global distribution and regional settings for your account. You can also change these settings after the account is created.

Geo-Redundancy ☐ Enable ☒ **Disable**

Multi-region Writes ☐ Enable ☒ **Disable**

[Review + create](#) [Previous](#) [Next: Networking](#) [Feedback](#)

- Click **Next**.
- On the **Networking** tab:
  - Select **Enable access from all networks**.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > Azure Cosmos DB > Create an Azure Cosmos DB account

## Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Basics Global distribution **Networking** Backup Policy Security Tags Review + create

### Network connectivity

You can connect to your Azure Cosmos DB account either publically, via public IP addresses or service endpoints, or privately, using a private endpoint.

Connectivity method \*

- ☒ All networks
- ☐ Public endpoint (selected networks)
- ☐ Private endpoint

All networks will be able to access this CosmosDB account. <http://aka.ms/network-security>

### Connection Security Settings

Minimum Transport Layer Security Protocol

Review + create Previous **Next: Backup Policy** Feedback

- Leave all settings as default and Click **Review + create**.
- This step created an Azure Cosmos DB account.

## Step 2 – Create a Database

- From the left pane, select **Data Explorer**.

Microsoft Azure

Search resources, services, and docs (G+/)

Copilot

cf-nancy-az@behalrites...  
MY DIRECTORY (BEHALRITESHG...)

Home > app-cosmos-db-01-acc

## app-cosmos-db-01-acc | Data Explorer

Azure Cosmos DB account

Search

Enable Azure Synapse Link Visual Studio Code

- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Quick start
- Data Explorer**
- Mirroring in Fabric
- Container Copy
- Resource visualizer
- Settings
- Integrations
- AI Capabilities (Preview)

+ New...

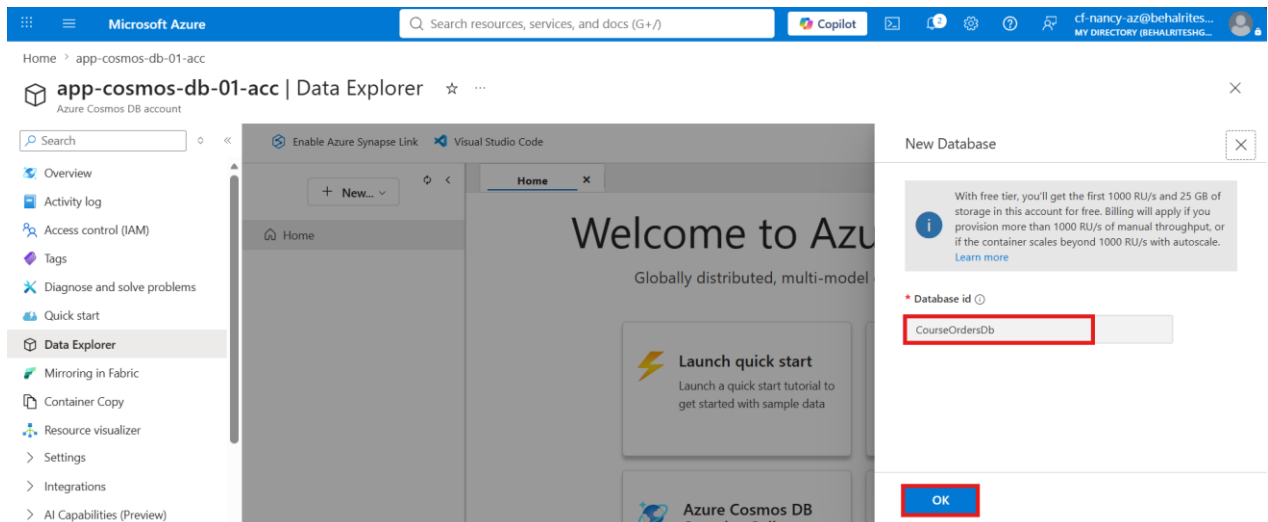
- + New Container
- + New Database**

## Welcome to Azure Cosmos DB

Globally distributed, multi-model database service for any scale

- Launch quick start**  
Launch a quick start tutorial to get started with sample data
- New Container**  
Create a new container for storage and throughput
- Azure Cosmos DB Sample Gallery**
- Connect**

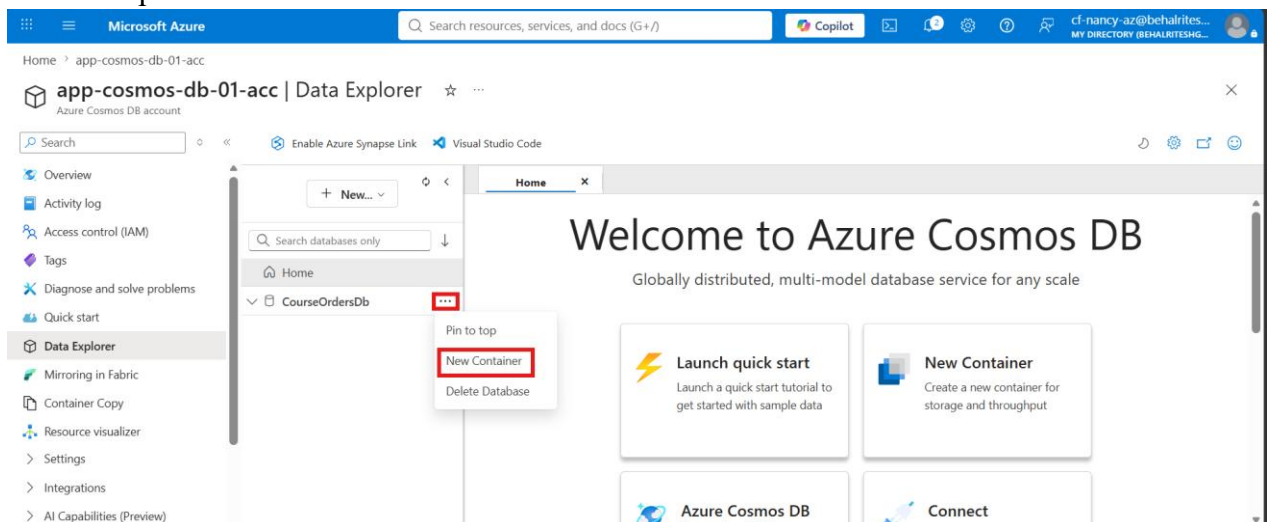
- Under the **NoSQL API**, click **New Database**.
- Enter the following details:
  - **Database ID:** CosmosOrdersDb



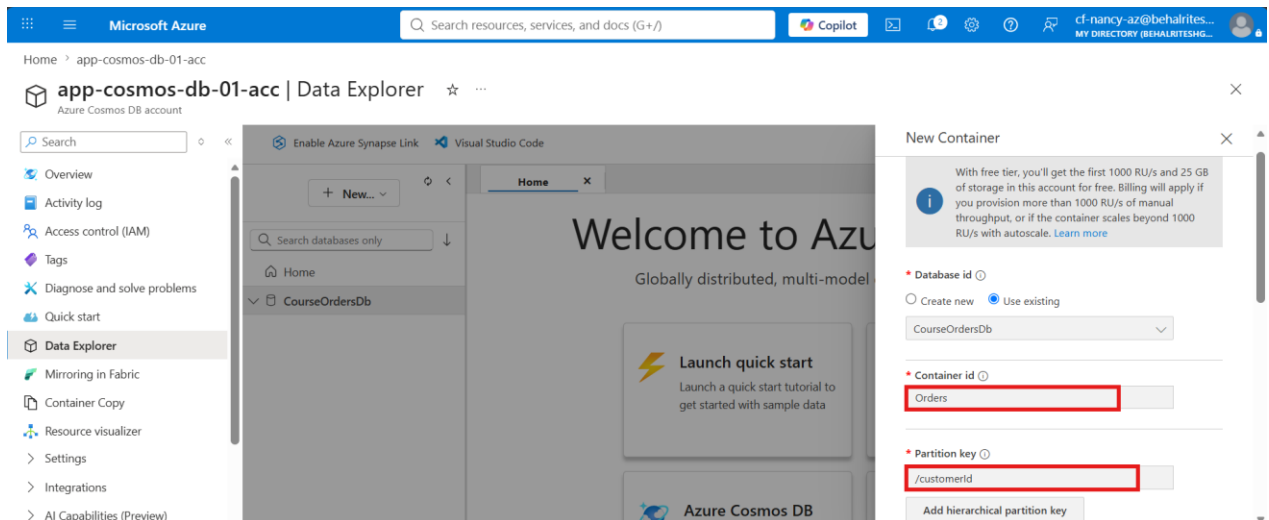
- Click **OK**.
- Verify that the database is created successfully.
- This step creates a database to organize containers and documents.

### Step 3 – Create a Container

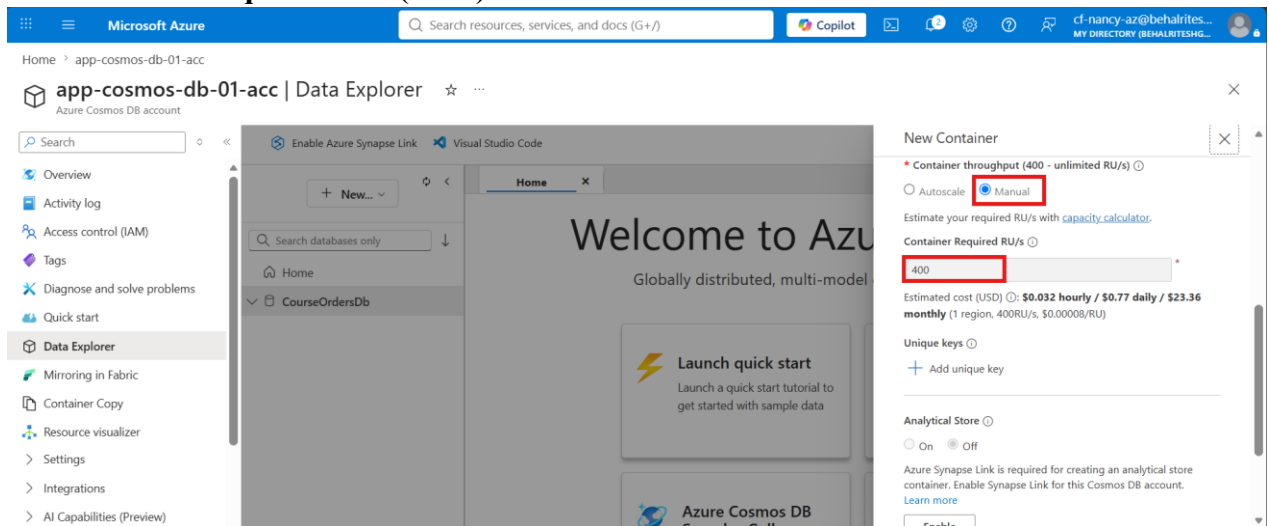
- Expand **CosmosOrdersDB**.



- Click **New Container**.
- Configure the following settings:
  - **Container ID:** Orders
  - **Partition Key:** /customerId



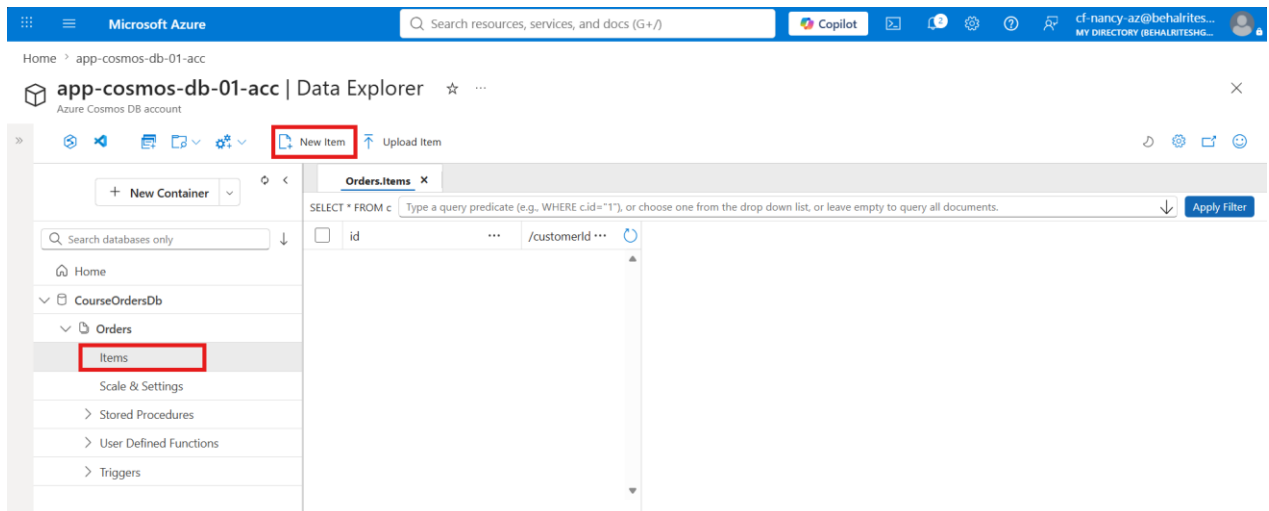
- **Container Throughput: Manual**
- **Request Units (RU/s): 400**



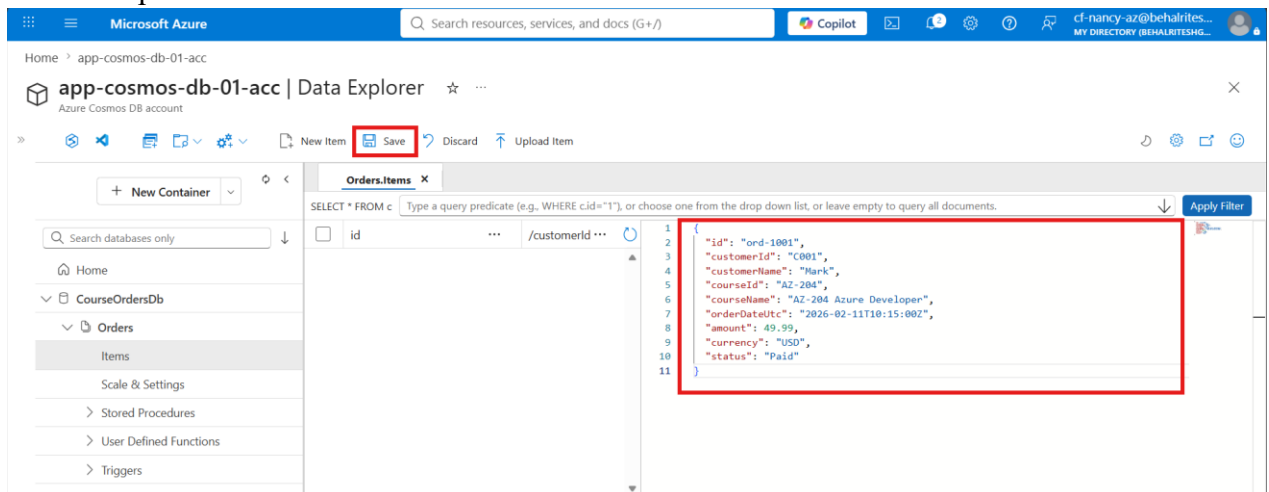
- Click **OK**.
- Verify that the **Orders** container is created.
- This step creates a container that stores JSON documents.

## Step 4 – Create the First Document

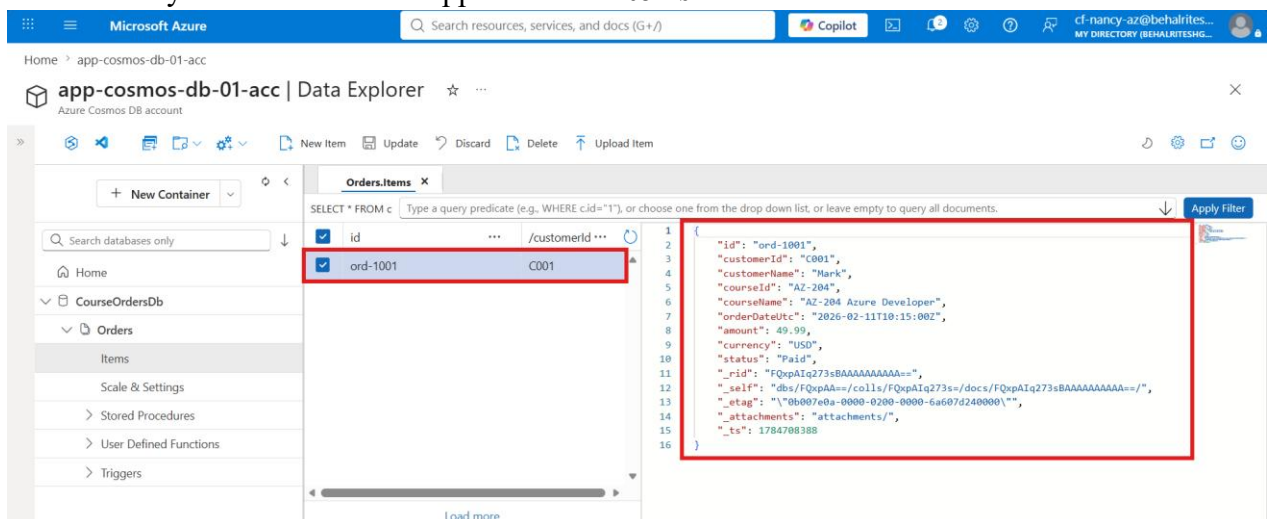
- Expand the **Orders** container.
- Select **Items**.



- Click **New Item**.
- Replace the default JSON with the first order document.



- Click **Save**.
- Verify that the document appears under **Items**.

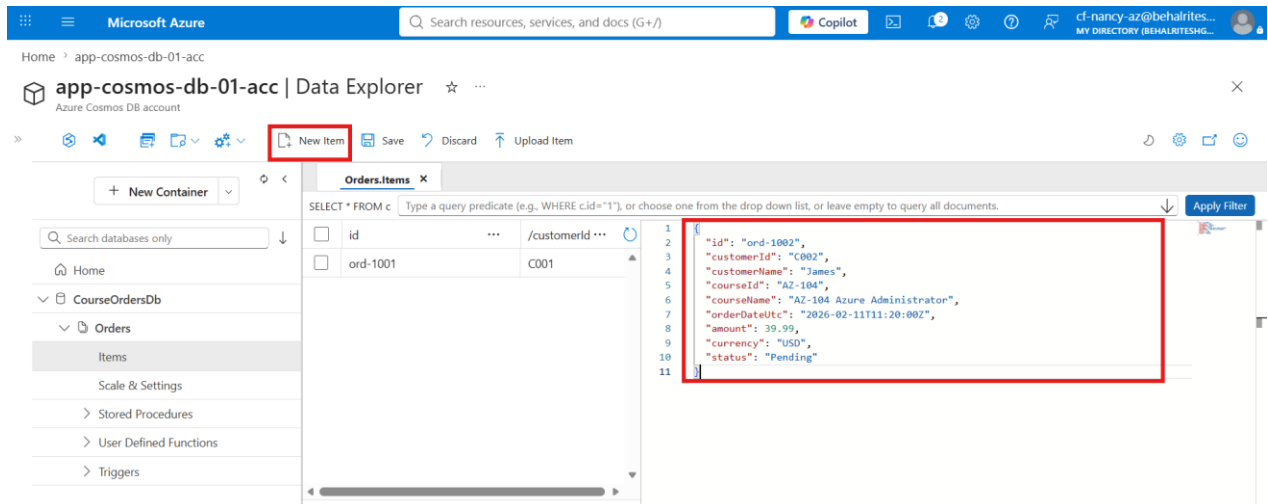




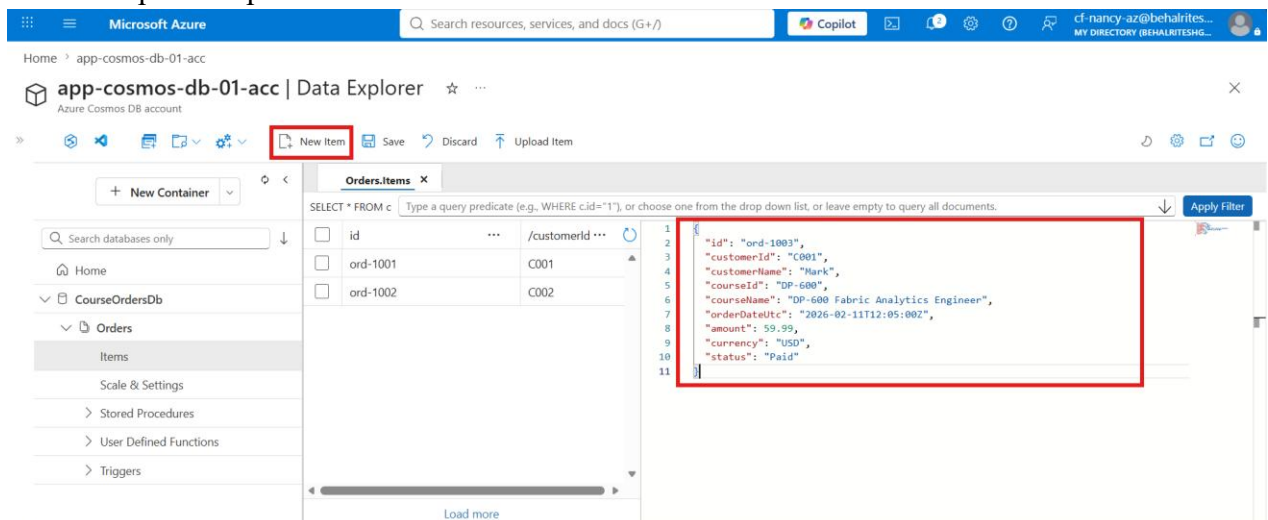
- Observe that Azure Cosmos DB automatically adds system-generated properties to the document.
- This step stores the first JSON document in the container.

## Step 5 – Create Additional Documents

- Click **New Item**.
- Add the second order document.



- Click **Save**.
- Repeat the process to add the third order document.



- Verify that all three documents are displayed under **Items**.

The screenshot shows the Azure Data Explorer interface for the 'app-cosmos-db-01-acc' account. The 'Orders.Items' container is selected, displaying a table with the following data:

| id       | /customerId |
|----------|-------------|
| ord-1001 | C001        |
| ord-1002 | C002        |
| ord-1003 | C001        |

The 'ord-1003' row is highlighted with a red box. The right pane shows the JSON document for 'ord-1003'.

```

1 {
2   "id": "ord-1003",
3   "customerId": "C001",
4   "customerName": "Mark",
5   "courseId": "DP-600",
6   "courseName": "DP-600 Fabric Analytics Engineer",
7   "orderDateUtc": "2026-02-11T12:05:00Z",
8   "amount": 59.99,
9   "currency": "USD",
10  "status": "Paid",
11  "_rid": "FQxpAIq273sEAAAAAAAAA==",
12  "_self": "dbs/FQxpAA==/colls/FQxpAIq273s=/docs/FQxpAIq273sEAAAAAAAAA==/",
13  "_etag": "\"0b001f41-0000-0200-0000-6a607e920000\"",
14  "_attachments": "attachments/",
15  "_ts": 1784708754
16 }
  
```

- Observe that documents having the same **CustomerId** are grouped using the configured partition key.
- This step stores multiple documents and demonstrates data partitioning in Azure Cosmos DB.

## Step 6 – Create a SQL Query

- Select New SQL Query.

The screenshot shows the Azure Data Explorer interface for the 'app-cosmos-db-01-acc' account. The 'New SQL Query' button is highlighted with a red box. The 'Orders.Items' container is still selected, and the table shows the same data as in the previous screenshot.

- Enter the following query:  
SELECT \* FROM c

Microsoft Azure | Search resources, services, and docs (G+/)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

Execute Query Save Query Download Query View

Search databases only

CourseOrdersDb

Orders

Items

Scale & Settings

Stored Procedures

User Defined Functions

Triggers

1 SELECT \* FROM c

Results Query Stats Index Advisor

1 - 3

```
{
  "id": "ord-1001",
  "customerId": "C001",
  "customerName": "Mark",
  "courseId": "AZ-204",
  "courseName": "AZ-204 Azure Developer",
  "orderDateUtc": "2026-02-11T10:15:00Z",
  "amount": 49.99,
  "currency": "USD",
  "status": "Paid",
  "_rid": "FQxpAIq273sBAAAAAAAAA==",
  "_self": "dbs/FQxpAA=/colls/FQxpAIq273s=/docs/FQxpAIq273sBAAAAAAAAA=/",
}
```

- Click **Execute Query**.
- Verify that all items stored in the container are displayed.
- Observe the **Query Stats** section to view the Request Units (RU) consumed by the query.

Microsoft Azure | Search resources, services, and docs (G+/)

Home > app-cosmos-db-01-acc

app-cosmos-db-01-acc | Data Explorer

Execute Query Save Query Download Query View

Search databases only

CourseOrdersDb

Orders

Items

Scale & Settings

Stored Procedures

User Defined Functions

Triggers

1 SELECT \* FROM c

2

Results Query Stats Index Advisor

| Metric                   | Value     |
|--------------------------|-----------|
| Request Charge           | 2.3 RUs   |
| Showing Results          | 1 - 3     |
| Retrieved document count | 3         |
| Retrieved document size  | 822 bytes |

Per-partition query metrics (CSV)

- This step retrieves all documents stored in the container using a SQL query.

## Step 6 – Filter Items Using a WHERE Clause

- Modify the query to filter documents by category or another field available in your data.
- Run:  

```
SELECT * FROM c
WHERE c.status = "Paid"
```

The screenshot shows the Microsoft Azure Data Explorer interface. The query editor displays the following SQL query:

```
1 SELECT * FROM courses c
2 WHERE c.status = "Paid"
```

The results pane shows a single document (1 - 2) with the following JSON structure:

```
{
  "customerId": "C001",
  "customerName": "Mark",
  "courseId": "AZ-204",
  "courseName": "AZ-204 Azure Developer",
  "orderDateUtc": "2026-02-11T10:15:00Z",
  "amount": 49.99,
  "currency": "USD",
  "status": "Paid",
  "_rid": "FQxpAIq273sBAAAAAAAAA==",
  "_self": "dbs/FQxpAA=/colls/FQxpAIq273s=/docs/FQxpAIq273sBAAAAAAAAA==/",
  "_etag": "\"0b007e0a-0000-0200-0000-6a607d240000\"",
  "_attachments": "attachments/",
  "_ts": 1784708388
}
```

- Click **Execute Query**.
- Verify that only the matching documents are returned.
- This step demonstrates how to filter documents using the **WHERE** clause.

## Step 7 – Use Multiple Filter Conditions

- Update the query to include multiple conditions.
- Run:

```
SELECT * FROM c
WHERE c.status = "Paid" AND c.courseId = "DP-600"
```

The screenshot shows the Microsoft Azure Data Explorer interface. The query editor displays the following SQL query:

```
1 SELECT * FROM c
2 WHERE c.status = "Paid" AND c.courseId = "DP-600"
```

The results pane shows a single document (1 - 1) with the following JSON structure:

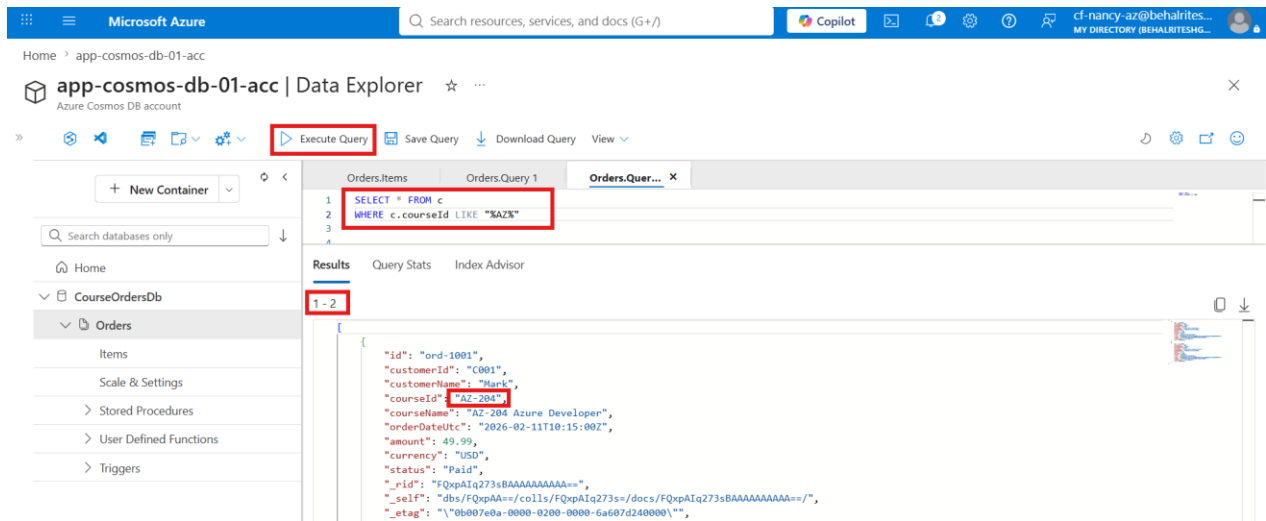
```
{
  "id": "ord-1003",
  "customerId": "C001",
  "customerName": "Mark",
  "courseId": "DP-600",
  "courseName": "DP-600 Fabric Analytics Engineer",
  "orderDateUtc": "2026-02-11T12:05:00Z",
  "amount": 59.99,
  "currency": "USD",
  "status": "Paid",
  "_rid": "FQxpAIq273sGAAAAAAAAA==",
  "_self": "dbs/FQxpAA=/colls/FQxpAIq273s=/docs/FQxpAIq273sGAAAAAAAAA==/",
  "_etag": "\"0b0076ad-0000-0200-0000-6a6082da0000\"",
  "_attachments": "attachments/"
}
```

- Click **Execute Query**.
- Verify that only the documents satisfying both conditions are displayed.
- This step demonstrates how to filter data using multiple conditions with the **AND** operator.

## Step 8 – Search Using the LIKE Operator

- Create a query to search for items containing specific text.

```
SELECT * FROM c
WHERE c.Name LIKE "%AZ%"
```



- Click **Execute Query**.
- Verify that only the matching documents are returned.
- This step demonstrates how to search documents using the **LIKE** operator.

## Verification

Verify the following:

- The Azure Cosmos DB account, database, and container are created successfully.
- Multiple JSON documents are stored in the Orders container.
- SQL queries return the expected documents.
- Filtered queries display only matching records.
- The LIKE query successfully searches documents based on the specified text.